

Rasporedjivanje nastave na PMF-u primjenom simuliranog kaljenja i Artificial Bee Colony algoritma

Tamir Kolade Oladejo

*Odsjek za matematičke i kompjuterske nauke
Prirodno-matematički fakultet, Univerzitet u Sarajevu
Sarajevo, Bosna i Hercegovina
tamir-kolade.oladejo@student.pmf.unsa.ba*

Abstract—Ovaj rad opisuje implementaciju sistema za automatsko rasporedjivanje nastave na Odsjeku za matematičke i kompjuterske nauke PMF-a UNSA. Problem je modeliran kao varijanta univerzitetskog rasporedjivanja nastave, pri čemu se ulazni CSV fajlovi pretvaraju u nastavne događaje koje treba dodijeliti terminima i učionicama. Prvo se generiše validan početni raspored koji poštuje tvrda ograničenja, a zatim se raspored poboljšava pomoću dva metaheuristička algoritma implementirana od početka: simulirano kaljenje i Artificial Bee Colony. Kvalitet rasporeda mjeri se fitness funkcijom koja strogo kažnjava nevalidna rješenja, a blaže kažnjava rupe, kasne termine, neravnomjernost i kršenje želja nastavnika. Eksperimenti su provedeni kroz 30 nezavisnih pokretanja svakog algoritma. Nakon optimizacije parametara, simulirano kaljenje je ostvarilo najbolji fitness 15590 i prosjek 16350.0, dok je Artificial Bee Colony ostvario najbolji fitness 16825 i prosjek 17167.5. Statistički testovi pokazuju da je razlika značajna na nivou 0.05, a veličina efekta je velika u korist simuliranog kaljenja.

Index Terms—university course timetabling, simulirano kaljenje, Artificial Bee Colony, metaheuristika, raspored nastave, fitness funkcija

I. UVOD

Rasporedjivanje nastave je kombinatorni optimizacioni problem u kojem se nastavni događaji moraju dodijeliti terminima, učionicama i nastavnicima. Problem je težak jer se istovremeno moraju zadovoljiti stroga fizička ograničenja, ali i poboljšati kvalitet rasporeda kroz mehka ograničenja. U projektnom zadatku ovaj problem je definisan kao rasporedjivanje nastave za Odsjek za matematičke i kompjuterske nauke PMF-a UNSA, uz obavezu korištenja dva metaheuristička algoritma i izradu PDF rasporeda po smjerovima [1].

Ovakav problem spada u širu porodicu University Course Timetabling Problem (UCTP), za koju se u literaturi često koriste heurističke i metaheurističke metode. Egzaktno pretraživanje kompletnog prostora rješenja nije praktično jer broj mogućih kombinacija brzo raste sa brojem predmeta, grupa, učionica i termina. Zbog toga se u ovom radu koriste simulirano kaljenje (SA) i Artificial Bee Colony (ABC), jer oba algoritma mogu raditi nad istom reprezentacijom rasporeda i postepeno popravljati postojeće rješenje.

Cilj rada je napraviti reproducibilan programski tok koji učitava podatke, generiše nastavne događaje, pravi početni

validan raspored, pokreće oba algoritma, statistički poredi rezultate i generiše konačni PDF raspored. Kod je implementiran modularno, ali bez korištenja biblioteka za gotovu metaheurističku optimizaciju, što je bio jedan od uslova projekta [1].

II. DEFINICIJA PROBLEMA

Neka je skup nastavnih događaja označen sa E , skup termina sa T , a skup učionica sa R . Jedno rješenje je raspored

$$S = \{(e_i, t_i, r_i) \mid e_i \in E, t_i \in T, r_i \in R\}, \quad (1)$$

gdje je svaki događaj dodijeljen tačno jednom terminu i jednoj učionici. Događaj predstavlja jedan blok nastave, npr. predavanje (P), auditorne vježbe (AV) ili laboratorijske vježbe (LV). Ako neki predmet ima fond od tri časa istog tipa nastave, taj fond se u projektu tretira kao jedan blok trajanja tri časa, a ne kao dva odvojena bloka. Time se izbjegava nerealan raspored u kojem se, na primjer, tri časa vježbi dijele na blok od dva časa i izolovan jedan čas u drugom terminu.

A. Tvrda ograničenja

Tvrda ograničenja definišu uslove bez kojih raspored nije validan. U implementaciji se provjeravaju sljedeća pravila:

- nastavnik ili asistent ne može držati dva časa u isto vrijeme;
- učionica ne može biti zauzeta sa dva događaja u istom terminu;
- ista studentska grupa, smjer, ciklus, godina i semestar ne mogu imati dva časa u isto vrijeme;
- učionica mora imati dovoljan kapacitet za broj studenata;
- laboratorijske vježbe moraju biti u laboratorijskoj učionici, uz posebno dozvoljen izuzetak za Programiranje I u ABG prema lokalnom modelu podataka;
- redovno zaposleni nemaju nastavu četvrtkom 13-14;
- nastava mora završiti najkasnije u 19:00, pa termin 19:00-20:00 nije dozvoljen.

U kodu se tvrda ograničenja tretiraju na dva načina. Prvo, početni raspored i operatori pomjeranja odbijaju poteze koji bi očigledno prekršili ograničenja. Drugo, fitness funkcija dodaje vrlo veliki penal za svaku hard constraint grešku koja

TABLE I
SAŽETAK ULAZNIH I GENERISANIH PODATAKA

Fajl / skup	Broj redova	Uloga u projektu
predmeti.csv	250	fond sati po smjerovima
osoblje.csv	225	zaduženja nastavnika/asistenata
studenti.csv	25	procjena broja studenata
ucionice.csv	12	kapacitet i lab oznaka
termini.csv	55	dostupni termini 08:00-19:00
ogranicenja.csv	46	hard i soft pravila
dogadjaji.csv	238	generisani nastavni blokovi

TABLE II
STRUKTURA GENERISANIH NASTAVNIH DOGADJAJA

Kategorija	Vrijednost	Broj dogadjaja
Tip nastave	P	119
Tip nastave	AV	79
Tip nastave	LV	40
Trajanje	1 sat	12
Trajanje	2 sata	161
Trajanje	3 sata	65
Ciklus	Bachelor	142
Ciklus	Master	96

eventualno ostane u rasporedu. Time se algoritmi usmjeravaju prema validnim rješenjima.

B. Mehka ograničenja

Mehka ograničenja ne čine raspored nevalidnim, ali utiču na kvalitet rješenja. U projektu se kažnjavaju rupe studentima, rupe nastavnicima, više od pet uzastopnih sati za studentsku grupu, neravnomjerna raspodjela nastave po danima, prerana nastava za master ciklus, vježbe prije predavanja, prerani termini za spoljne saradnike i individualne želje iz fajla ograničenja. Ova pravila su uvedena jer studentski i nastavnički raspored nije dovoljno samo učiniti validnim; on treba biti i upotrebljiv.

III. PODACI

Ulazni podaci su strukturirani u CSV fajlove. Projekat koristi fajlove za predmete, osoblje, učionice, termine, procjenu broja studenata i ograničenja. Na osnovu njih generator dogadjaja formira listu konkretnih nastavnih blokova koje treba rasporediti.

Procjena broja studenata nije uzeta kao tačan službeni broj, nego kao radna aproksimacija potrebna za kapacitete učionica i dijeljenje laboratorijskih vježbi. Najveća grupa u podacima je Kompjuterske nauke, Bachelor, I godina, sa 105 studenata. Zbog toga se veliki zajednički predmeti moraju rasporediti u veće sale, posebno u ABG. Laboratorijske vježbe se dodatno dijele u grupe kada broj studenata prelazi kapacitet najveće laboratorijske učionice.

TABLE III
KOMPONENTE FITNESS FUNKCIJE

Komponenta	Težina	Obrazloženje
Hard constraint greške	1 000 000	nevalidan raspored mora biti skoro uvijek lošiji od validnog
Nerasporedjeni dogadjaji	100 000	svaki čas mora biti prisutan u rasporedu
Rupe studentima	20	rupe direktno pogoršavaju studentski raspored
Previše uzastopnih sati	50	dugi blokovi su naporni i nepraktični
Neravnomjernost po danima	15	nastava ne treba biti koncentrisana u jedan dan
Master prije 14h	30	master nastava se preferira kasnije
Vježbe prije predavanja	40	pedagoški je bolje da predavanje prethodi vježbama
Rupe nastavnicima	10	manje važno od rupa studentima, ali i dalje nepoželjno
Spoljni saradnik prije 16h	50	spoljni saradnici često imaju radne obaveze ranije
Individualne želje	15	želje su korisne, ali nisu obavezne

IV. REPREZENTACIJA RJEŠENJA I FITNESS FUNKCIJA

Rješenje je lista objekata tipa *StavkaRasporeda*. Svaka stavka čuva identifikator dogadjaja, predmet, tip nastave, nastavnika, smjerove, ciklus, godinu, grupu, broj studenata, učionicu, dan, početak, kraj i trajanje. Ovakva reprezentacija je jednostavna za validaciju jer se svaka provjera može svesti na poređenje zauzetosti po nastavniku, učionici i studentskom ključu.

Fitness funkcija minimizira ukupni penal:

$$F(S) = \sum_{i=1}^k w_i c_i(S), \quad (2)$$

gdje je $c_i(S)$ broj prekršaja ili nepoželjnih pojava u rasporedu, a w_i težina te komponente. Manja vrijednost fitnessa znači bolji raspored.

Početni greedy raspored imao je fitness 19065. U njemu nije bilo hard constraint grešaka niti nerasporedjenih dogadjaja, ali su soft penali pokazali prostor za optimizaciju. Najveći doprinos početnom fitnessu imale su rupe studentima (335 slučajeva, penal 6700), više od pet uzastopnih sati studentima (84 slučaja, penal 4200), vježbe prije predavanja (52 slučaja, penal 2080) i master nastava prije 14h (59 slučajeva, penal 1770). Time fitness funkcija jasno razdvaja validnost rasporeda od njegove praktične upotrebljivosti.

V. ALGORITAM 1: SIMULIRANO KALJENJE

Simulirano kaljenje je metaheuristika inspirisana procesom fizičkog kaljenja materijala. Algoritam kreće od validnog početnog rasporeda. U svakoj iteraciji nasumično se bira jedan dogadjaj i pokušava premjestiti u drugi validan termin i učionicu. Ako novo rješenje ima bolji fitness, prihvata se. Ako je lošije, može se prihvatiti sa vjerovatnoćom

$$P = \exp \left(-\frac{F(S') - F(S)}{T} \right), \quad (3)$$

gdje je T trenutna temperatura. Temperatura se postepeno smanjuje faktorom hladjenja. Na početku algoritam može prihvatati i lošija rješenja, što smanjuje rizik od zapinjanja u lokalnom minimumu. Kasnije postaje konzervativniji.

```

1:  $S \leftarrow$  početni raspored
2:  $S^* \leftarrow S$ 
3:  $T \leftarrow 500$ 
4: for  $i = 1$  do 600 do
5:    $S' \leftarrow$  validan susjed rasporeda  $S$ 
6:   if  $F(S') < F(S)$  then
7:      $S \leftarrow S'$ 
8:   else
9:     prihvati  $S'$  sa vjerovatnoćom  $\exp(-(F(S') - F(S))/T)$ 
10:  end if
11:  if  $F(S) < F(S^*)$  then
12:     $S^* \leftarrow S$ 
13:  end if
14:   $T \leftarrow 0.99T$ 
15: end for
16: return  $S^*$ 

```

U projektu je korišteno 600 iteracija, početna temperatura 500 i faktor hladjenja 0.99. Ove vrijednosti su odabrane nakon probnog podešavanja jer omogućavaju dovoljno istraživanja u početku, ali brže prelaze u strožiju lokalnu optimizaciju pred kraj pokretanja.

VI. ALGORITAM 2: ARTIFICIAL BEE COLONY

Artificial Bee Colony je populaciona metaheuristika inspirisana ponašanjem pčela pri traženju hrane. U ovom projektu jedan izvor hrane predstavlja jedan kandidatski raspored. Kvalitet izvora je obrnuto povezan sa fitnessom, jer je manji fitness bolji.

Algoritam koristi tri faze. U fazi zaposlenih pčela svako rješenje pokušava napraviti susjedno rješenje pomjeranjem nastavnog događaja. U fazi posmatračkih pčela bolji izvori imaju veću vjerovatnoću da budu izabrani za dodatno popravljivanje. U fazi izviđačkih pčela izvor koji se dugo ne poboljšava zamjenjuje se novim perturbovanim rasporedom.

```

1: generiši populaciju od 4 izvora hrane
2: zapamti najbolji izvor
3: for  $c = 1$  do 70 do
4:   for svaki izvor do
5:     probaj napraviti bolji susjedni raspored
6:   end for
7:   for svaka posmatračka pčela do
8:     izaberi izvor proporcionalno kvalitetu
9:     probaj napraviti bolji susjedni raspored
10:  end for
11:  zamijeni izvore bez poboljšanja duže od limita 15
12:  ažuriraj najbolji raspored
13: end for
14: return najbolji raspored

```

ABC je podešen na 70 ciklusa, 4 izvora i limit 15. Broj izvora je zadržan relativno mali da bi vrijeme izvršavanja ostalo uporedivo sa SA algoritmom, dok je broj ciklusa povećan kako bi ABC imao više prilika za poboljšanje populacije.

TABLE IV
KORIŠTENI HIPERPARAMETRI NAKON OPTIMIZACIJE

Parametar	Vrijednost
Broj nezavisnih pokretanja po algoritmu	30
SA broj iteracija po pokretanju	600
SA početna temperatura	500.0
SA faktor hladjenja	0.99
ABC broj ciklusa po pokretanju	70
ABC broj izvora hrane	4
ABC limit	15
Najkasniji kraj nastave	19:00

TABLE V
DESKRIPTIVNA STATISTIKA ZAVRŠNOG FITNESSA

Alg.	Najbolji	Najgori	Prosjek	Medijan	Std.
SA	15590	17215	16350.00	16342.50	424.39
ABC	16825	17505	17167.50	17222.50	185.68

VII. EKSPERIMENTALNA POSTAVKA

Eksperimenti su provedeni na istoj instanci rasporeda i istom validnom početnom rasporedu. Svaki algoritam je pokrenut 30 puta sa različitim seed vrijednostima, što je minimalni broj pokretanja tražen projektnim zadatkom [1]. Mjeren je najbolji završni fitness, broj hard constraint grešaka i vrijeme izvršavanja po pokretanju. Vrijeme je mjereno u lokalnom Python okruženju pomoću sistemskog vremena, pa apsolutne vrijednosti zavise od računara, dok je relativno poređenje korisno jer su oba algoritma pokrenuta u istom okruženju.

Početni raspored imao je 238 rasporedjenih događaja, 0 nerasporedjenih događaja i 0 hard constraint grešaka. Njegov fitness iznosio je 19065. Nakon optimizacije, najbolji SA raspored imao je fitness 15590, a najbolji ABC raspored 16825. Oba najbolja rasporeda su imala 0 hard constraint grešaka. Konačni PDF raspored generisan je iz najboljeg SA rješenja jer je ono ostvarilo najmanji fitness.

VIII. REZULTATI I DISKUSIJA

Tabela V prikazuje deskriptivnu statistiku završnih fitness vrijednosti. Budući da se fitness minimizira, niža vrijednost je bolja.

SA je ostvario bolji najbolji rezultat, bolji prosjek i bolji medijan. Najbolji SA rezultat poboljšao je početni fitness sa 19065 na 15590, što je smanjenje od 3475 fitness jedinica, odnosno približno 18.2%. U prosjeku, SA je ostvario fitness 817.5 jedinica manji od ABC algoritma, što predstavlja oko 4.8% bolji prosječni rezultat u odnosu na ABC prosjek. Najbolji SA rezultat je za 1235 fitness jedinica bolji od najboljeg ABC rezultata, odnosno približno 7.3%.

ABC je imao manju standardnu devijaciju, što znači da su njegovi rezultati bili stabilniji i koncentrisaniji. Ipak, u ovoj konfiguraciji stabilnost ABC-a nije dovela do boljeg fitnessa. SA je pokazao veću varijabilnost, ali je pronalazio

TABLE VI
VRIJEME IZVRŠAVANJA PO POKRETANJU

Alg.	Prosjek	Std.	Min	Max	Medijan
SA	27.20 s	3.29 s	24.99 s	38.07 s	25.56 s
ABC	25.25 s	2.40 s	23.40 s	31.65 s	24.19 s

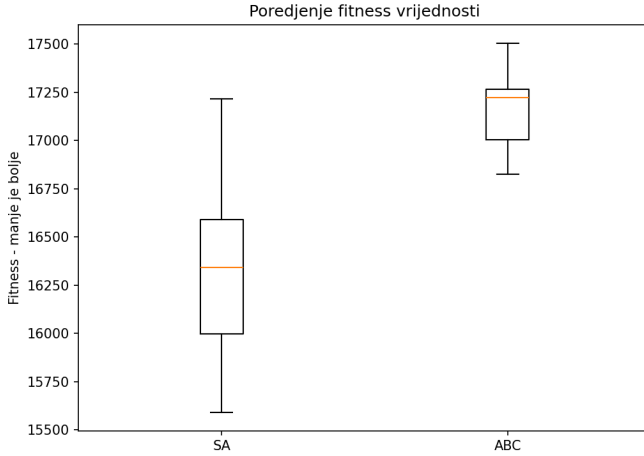


Fig. 1. Distribucija završnih fitness vrijednosti za 30 pokretanja SA i ABC.

znatno kvalitetnija rješenja. To je očekivano nakon promjene temperature i hlađenja: SA brže smanjuje temperaturu i pred kraj više čuva dobra rješenja, dok ABC sa samo 4 izvora ima ograničenu populacionu raznolikost.

Vrijeme izvršavanja pokazuje da su algoritmi nakon podešavanja vremenski uporedivi. ABC je u prosjeku bio oko 1.96 sekundi brži po jednom pokretanju, ali je razlika mala u odnosu na razliku u kvalitetu rješenja. Zbog toga je za konačni raspored odabran SA rezultat.

Za statističku provjeru prvo je urađena Shapiro-Wilk provjera normalnosti. Za SA je dobijeno $p = 0.7435$, a za ABC $p = 0.2186$, pa nema dovoljno dokaza da se odbaci normalnost na nivou 0.05. Zbog toga je za uparene rezultate primijenjen parni t-test, koji je dao $p < 0.001$. Kao dodatna robusna provjera, Wilcoxon signed-rank test dao je $p < 0.001$, a permutacijski test iz programa dao je $p = 0.001$. Svi rezultati vode istom zaključku: razlika je statistički uočljiva na nivou $\alpha = 0.05$.

Praktična veličina efekta je velika. Upareni Cohenov d_z iznosi približno -1.81 ako se računa na razlikama SA-ABC; negativan predznak znači da SA ima manji, odnosno bolji fitness. Ako se koristi pooled Cohenov d za razliku prosjeka ABC-SA, dobija se približno 2.50, što također ukazuje na veliki efekat u korist SA algoritma. Cliff's delta iznosi približno 0.92 ako se računa kao vjerovatnoća da SA daje manji fitness minus vjerovatnoća da ABC daje manji fitness. To znači da SA veoma često nadmašuje ABC u direktnom poredjenju završnih fitness vrijednosti.

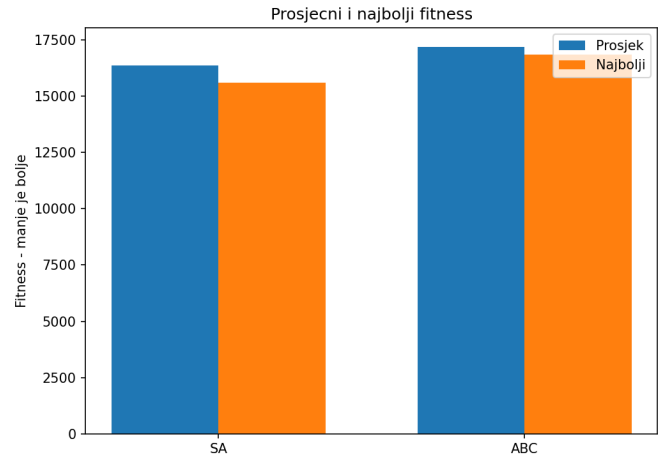


Fig. 2. Poredjenje prosječnog i najboljeg fitnessa.

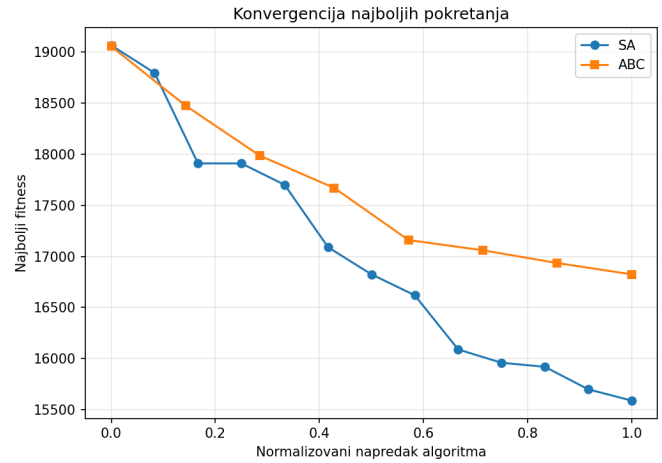


Fig. 3. Konvergencija najboljih pokretanja. X-osa je normalizovana jer SA i ABC imaju različite skale iteracija i ciklusa.

IX. ZAKLJUČAK

Implementirani sistem uspješno generiše validan raspored nastave iz CSV podataka, zatim poredi simulirano kaljenje i Artificial Bee Colony na istoj instanci problema. Oba algoritma su implementirana od početka i rade nad istom reprezentacijom rješenja. Konačni rasporedi nemaju hard constraint greške i pokrivaju svih 238 generisanih nastavnih dogadjaja.

Nakon optimizacije parametara, simulirano kaljenje je jasno ostvarilo bolje rezultate od ABC algoritma. SA je imao bolji najbolji fitness, bolji prosjek, bolji medijan i statistički značajnu prednost prema parnom t-testu, Wilcoxon testu i permutacijskom testu. ABC je ostao nešto brži i stabilniji po standardnoj devijaciji, ali ta stabilnost nije bila dovoljna da nadoknadi slabiji kvalitet rješenja. Zbog toga se za ovu instancu rasporedjivanja nastave može zaključiti da je SA pogodniji izbor kada je cilj dobiti što bolji raspored, dok je ABC koristan kao brža i stabilnija, ali slabija alternativa.

TABLE VII
STATISTIČKI TESTOVI I EFFECT SIZE

Mjera	Vrijednost	Tumačenje
Shapiro SA	0.7435	normalnost se ne odbacuje
Shapiro ABC	0.2186	normalnost se ne odbacuje
Parni t-test	< 0.001	statistički značajna razlika
Wilcoxon	< 0.001	robustna potvrda istog zaključka
Permutacijski test	0.001	test iz programa
Cohenov d_z	-1.81	veliki efekat u korist SA
Pooled Cohenov d	2.50	velika razlika prosjeka
Cliff's delta	0.92	SA često daje manji fitness

Najveće ograničenje rada je oslanjanje na procijenjene brojeve studenata i ručno definisane težine fitness funkcije. U budućem radu mogla bi se dodati automatska kalibracija težina, veći broj perturbacionih operatora, lokalna pretraga nakon metaheuristike i preciznije modeliranje izbornih predmeta i stvarnih studentskih upisa.

PRIOLOG: AI TOOLS DISCLOSURE

Tokom izrade projekta i seminarskog rada korišten je ChatGPT kao pomoćni alat. Alat je korišten za brainstorming strukture projekta, objašnjenje metaheuristika, pronalaženje i ispravljanje grešaka u kodu, pojednostavljivanje implementacije, formatiranje seminarskog rada i provjeru da li rad prati zadate projektne instrukcije. Tipičan prompt bio je: "Napravi mi seminarski rad po datom templateu i uputstvima koristeći podatke iz zipovanog projekta". Odgovor AI alata je korišten kao nacrt koji je prilagodjen stvarnim rezultatima iz CSV fajlova i izvještaja projekta.

Kritička refleksija: AI je značajno pomogao u organizaciji rada, objašnjavanju i implementaciji algoritama, identifikaciji redundantnog koda i parsiranju podataka. Ipak, više puta je bilo potrebno ručno provjeriti da li kod stvarno radi, da li su svi događaji rasporedjeni, da li PDF raspored ima ispravan format i da li su rezultati eksperimenata preuzeti iz stvarnih izlaznih fajlova. Student ostaje odgovoran za tačnost koda, eksperimentalnih brojeva i tumačenja rezultata.

REFERENCES

- [1] S. Delalić, "Projektni zadatak – Uvod u vještačku inteligenciju: Rasporedjivanje nastave na PMF-u primjenom metaheurističkih algoritama," HackMD, 2026. [Online]. Available: <https://hackmd.io/@rbPCUpkvQu-ncdj00mkqtQ/ryxISPDbx>